Wer schlägt die Messlatte?

Im Ordner modelle/ treten Ridge, Random Forest und XGBoost gegen die Messlatte aus Heft 2 an. Zwei Dateien führen den Vergleich, eine erklärt, welche Merkmale die Vorhersage tragen, und drei prüfen, wie belastbar das Ergebnis ist. Dieses Heft grenzt die Dateien voneinander ab und nennt jede Funktion mit ihrem Zweck.

Ordner: modelle/
Umfang: 7 Dateien, 68 Funktionen
Anschluss an: Heft 1 und 2 und Band 1 und 2 der Laienfassung
Grundlage: der Quelltext im Abgabeordner, Stand 08.10.2026

Heft 1
prep/ und tests/

Heft 2
vorpruefung/

Heft 3
modelle/

TEIL 1

Der Ordner auf einen Blick

> Hier fällt die Antwort auf die Forschungsfrage: Welches Verfahren sagt besser vorher als die Messlatte, und lohnt sich der Aufwand?

Der Ordner hat einen Kern und einen Rand. Der Kern sind die beiden Vergleichsdateien: `m02_menge.py` für die Frage „Wie viele?“, `m03_struktur.py` für „Welche Art?“. Beide lassen die Verfahren gegen die Messlatte aus Heft 2 antreten, auf denselben Zeilen und in denselben Prüfgruppen. Der Rand sind vier Dateien, die den Vergleich erklären oder absichern.

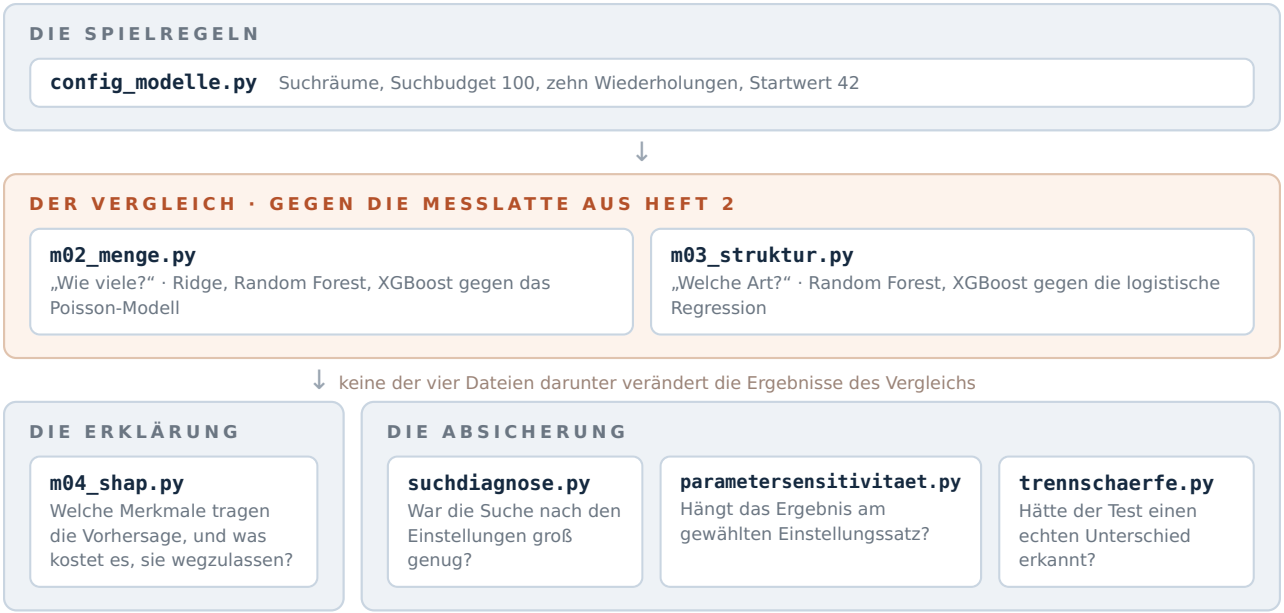


Abbildung 1: Der Kern des Ordners sind die beiden Vergleichsdateien, für jede Frage der Arbeit eine. Von den vier Dateien darunter erklärt eine die Ergebnisse, drei prüfen, wie belastbar sie sind.

DATEI	DIE FRAGE, DIE SIE BEANTWORTET	FUNKTIONEN
`config_modelle.py`	Hält fest, was beim Rechnen gilt.	0
`m02_menge.py`	Vergleich für „Wie viele?“: drei Verfahren, 300 Läufe.	18
`m03_struktur.py`	Vergleich für „Welche Art?“: zwei Verfahren, 100 Läufe.	21
`m04_shap.py`	Welche Merkmale tragen die Vorhersage?	11
`parametersensitivitaet.py`	Hängt das Ergebnis am gewählten Einstellungssatz?	9
`suchdiagnose.py`	War das Suchbudget groß genug?	6
`trennschaerfe.py`	Hätte der Test einen echten Unterschied erkannt?	3

TEIL 2

Wie sich die Dateien voneinander abgrenzen

> Zwei Dateien rechnen den Vergleich, vier erklären oder prüfen ihn. Die Grenzen verlaufen zwischen den beiden Fragen der Arbeit und zwischen Rechnen, Erklären und Absichern.

Menge und Struktur: zweimal dasselbe Muster

`m03_struktur.py` ist ein Spiegel von `m02_menge.py`: dieselben Phasen, dieselbe Reihenfolge, aber eigene Fallstricke. Beide arbeiten in vier Phasen und hängen auf Wunsch eine fünfte an. Einige kleine Funktionen stehen deshalb wortgleich in beiden Dateien. Das ist Absicht: Jede der beiden soll allein lauffähig bleiben.

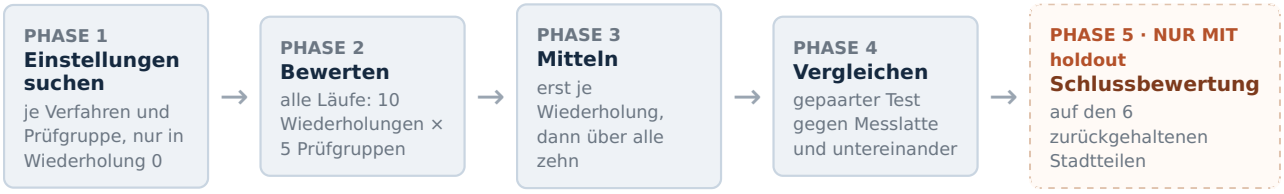


Abbildung 2: Das Muster, nach dem `m02_menge.py` und `m03_struktur.py` gebaut sind. Beide Dateien haben dieselben Phasen-Funktionen in derselben Reihenfolge, dazu einige eigene Hilfsfunktionen. Phase 5 läuft nur mit dem Zusatz `holdout`. Dass sie nur einmal gefahren wird, ist eine Arbeitsregel.

Die Unterschiede liegen in der Frage, nicht im Ablauf. `m02` sagt eine Zahl vorher und misst, wie weit es daneben liegt. `m03` sagt eine von vier Einsatzarten vorher und misst, wie gut es alle vier trifft, auch die seltenen.

m02_menge.py · „WIE VIELE?“

Zielgrößen	Zahl der Einsätze, Einsätze je 1.000 Einwohner
Verfahren	Ridge, Random Forest, XGBoost
Messlatte	Poisson-Modell
Gütemaß	RMSE: je kleiner, desto besser
Läufe	2 × 3 × 10 × 5 = 300
Besonderheit	Fehlerfunktionen für Zähldaten, Holm-Korrektur beim Vergleich der Verfahren untereinander

m03_struktur.py · „WELCHE ART?“

Zielgröße	überwiegende Einsatzart, vier Klassen
Verfahren	Random Forest, XGBoost
Messlatte	multinomiale logistische Regression
Gütemaß	Macro-F1: je größer, desto besser
Läufe	1 × 2 × 10 × 5 = 100
Besonderheit	Klassengewichte für seltene Einsatzarten, feste Übersetzung der Klassen in Zahlen

Abbildung 3: Zwei Dateien, ein Bauplan. Ridge fehlt im Strukturstrang, weil seine Variante für Klassen Werte unter null oder über eins liefert, also keine Wahrscheinlichkeiten.

Erklären gegen Absichern

`m04_shap.py` fragt, welche Merkmale eine Vorhersage tragen. Die drei übrigen Dateien fragen, wie belastbar der Vergleich selbst ist, und zwar jede aus einer anderen Richtung: `suchdiagnose.py` prüft die Suche nach den Einstellungen, `parametersensitivitaet.py` die Wahl zwischen den gefundenen Einstellungen,

`trennschaerfe.py` den statistischen Test am Ende. Keine der vier verändert die Ergebnisse von `m02` und `m03`.

Zwei Einstellungsdateien

`config_modelle.py` ist das Gegenstück zu `prep/config.py` aus Heft 1. Dort steht, was in die Dateien geschrieben wird, hier nur, was beim Rechnen gilt. Merkmale, Einsatzarten und die Zahl der Prüfgruppen stehen deshalb weiter in `prep/config.py`: Die Modellskripte lesen diese Festlegungen, sie treffen sie nicht.

DREI SCHUTZREGELN, IN BEIDEN STRÄNGEN GLEICH

Erstens muss auch die Suche nach den Einstellungen nach Stadtteilen trennen. Sonst lägen dieselben Stadtteile mit ihren 132 Monaten in innerem Training und innerer Prüfung, und die Suche wäre zu optimistisch (Funktion `tune`). **Zweitens** wird zweistufig gemittelt, weil die 50 Läufe dieselben 30 Stadtteile in wechselnder Mischung sind (Funktion `aggregiere`). **Drittens** sortiert `main()` das Hold-out immer gleich zu Beginn aus. Nur mit dem Zusatz `holdout` bekommt ganz am Ende allein die Schlussbewertung die sechs Stadtteile zu sehen.

AUFMERKSAMKEIT IST NICHT WERT

`m04_shap.py` beantwortet die Merkmalsfrage auf zwei Wegen. Die *Attribution* misst, wie ein Modell seine Aufmerksamkeit auf die Merkmale verteilt. Die *Ablation* misst, was es kostet, eine Merkmalsgruppe wegzulassen. Ein Merkmal kann viel Aufmerksamkeit binden und trotzdem ersetzbar sein, wenn ein anderes dieselbe Information trägt.

TEIL 3

Datei für Datei

> Für jede Datei: was sie tut, was sie liest und schreibt, und eine Tabelle mit jeder Funktion und ihrem Zweck. Die Reihenfolge folgt dem Ablauf.

SO LIEST MAN DIE TABELLEN

Eine Python-Datei ist eine Art Rezept. Eine *Funktion* ist ein einzelner, benannter Arbeitsschritt darin: Sie bekommt etwas hinein, tut etwas damit und gibt etwas zurück. Funktionen, deren Name mit einem Unterstrich beginnt (etwa `_holm`), sind als interne Hilfsschritte gedacht. Einige davon leihen sich andere Dateien trotzdem aus. Die Funktion `main` oder `run` ist der *Startpunkt*: Sie läuft, wenn man die Datei aufruft, und ruft ihrerseits die übrigen Funktionen in der richtigen Reihenfolge auf.

In den folgenden Tabellen steht jede Funktion einzeln, in der Reihenfolge, in der sie im Quelltext vorkommt. Die Markierung *Startpunkt* zeigt, wo der Ablauf beginnt, *Hilfsfunktion* einen kleinen Zuarbeiter, *innen* eine Funktion, die nur innerhalb einer anderen Funktion existiert. Die Funktionszahlen in den Übersichten zählen solche inneren Funktionen mit.

---

modelle/

config_modelle.py

Die Spielregeln des Vergleichs

Suchräume, Suchbudget, Wiederholungen und Startwert.

LIEST	SCHREIBT	BENUTZT VON
nichts	nichts	m02, m03, m04, parametersensitivitaet, suchdiagnose sowie v0, v1, v3, v4, panelprofil

Wie `prep/config.py` enthält diese Datei keine Funktion, nur Festlegungen. Darin stehen die Suchräume, also die Bereiche, in denen nach den Einstellungen gesucht wird: für Ridge eine Einstellschraube (die Stärke der Bremse), für den Random Forest vier, für XGBoost sieben. Jedes Verfahren bekommt dasselbe Suchbudget von 100 Ziehungen.

Der Startwert 42 macht jeden Zufallsschritt wiederholbar, und `WIEDERHOLUNGEN = 10` legt fest, wie oft die Bewertung mit neu gemischten Prüfgruppen läuft. Die Messlatte steht hier nicht: Sie hat keine Einstellschraube, es gibt nichts zu suchen.

Keine Funktionen, nur Festlegungen.

modelle/

m02_menge.py

Der Vergleich für „Wie viele?“

Ridge, Random Forest und XGBoost gegen das Poisson-Modell.

LIEST
`regression.parquet` und die Läufe der Messlatte aus v1, dazu `klassifikation.parquet` für die Zuteilung der Prüfgruppen

SCHREIBT
`results/regression/`: `tuning.csv`, `menge_folds.csv`, `menge_mittel.csv`, `vergleich.csv`, `leakage_diagnose.csv`, Vorhersagen, mit `holdout` auch `holdout.csv`

Zwei Zielgrößen, drei Verfahren, zehn Wiederholungen, fünf Prüfgruppen: Das ergibt 300 Läufe. Die Datei arbeitet sie in vier Phasen ab und hängt mit dem Zusatz `holdout` eine fünfte an, die einmalige Schlussbewertung. Gegner ist die Stufe-2-Messlatte, nicht der bloße Durchschnitt.

FUNKTION	WOZU SIE DA IST
BAUEN UND SUCHEN	
`verfahren()`	Baut das noch nicht eingestellte Modell. Ridge bekommt eine Standardisierung der Merkmale und rechnet auf log(1+y), der Random Forest eine Poisson-Fehlerfunktion, XGBoost eine Tweedie-Fehlerfunktion. Beide passen zu Zähldaten.
`suchraum()`	Übersetzt die Suchräume aus `config_modelle.py` in Zufallsverteilungen, aus denen die Suche ihre Ziehungen nimmt.
`tune()`	Sucht mit 100 zufälligen Ziehungen die besten Einstellungen auf den Übungsstadtteilen einer Prüfgruppe. Auch die innere Aufteilung trennt dabei nach Stadtteilen. Zurückgegeben werden die Einstellungen, nicht das Modell.
EIN EINZELNER LAUF	
`ein_lauf()`	Ein Training und eine Vorhersage mit Stoppuhr, einmal auf einem Rechenkern und einmal auf allen. Liefert Fehlermaße, Laufzeiten, den Extrapolationsanteil und ob Ridge Werte unter null vorhergesagt hat.
`extrapolationsanteil()`	Anteil der Prüfzeilen, die bei mindestens einem Merkmal außerhalb des im Training gesehenen Wertebereichs liegen.
PHASE 1 · EINSTELLUNGEN SUCHEN	
`phase_tuning()`	Sucht für jedes Verfahren und jede Prüfgruppe einmal die Einstellungen, nur in Wiederholung 0, und schreibt sie nach `tuning.csv`. Beide Zielgrößen bekommen denselben Satz.
`_rein_python()` Hilfsfunktion	Wandelt NumPy-Zahlen in gewöhnliche Python-Zahlen um, damit die Einstellungen sauber gespeichert und wieder eingelesen werden.
`_parameter_je_fold()` Hilfsfunktion	Macht aus `tuning.csv` eine Nachschlagetabelle: Zielgröße, Verfahren und Prüfgruppe ergeben die Einstellungen.

FUNKTION	WOZU SIE DA IST
PHASE 2 · BEWERTEN	
phase_bewertung()	Trainiert in allen 300 Läufen ein frisches Modell mit den gefundenen Einstellungen und bewertet es auf den Prüfstadtteilen.
PHASE 3 · MITTELN	
aggregiere()	Mittelt zweistufig, erst je Wiederholung über die fünf Prüfgruppen, dann über die zehn Wiederholungen. Maßgeblich ist die Streuung der zehn Wiederholungsmittel.
PHASE 4 · VERGLEICHEN	
_holm() Hilfsfunktion	Holm-Korrektur: verschärft die Grenze für Signifikanz, wenn mehrere Tests zu einer Familie gehören.
_gepaart() Hilfsfunktion	Gepaarter Wilcoxon-Test zwischen zwei Wertereihen, dazu mittlere Differenz, Konfidenzintervall und die Zahl der gewonnenen Läufe.
vergleiche()	Jedes Verfahren gegen die Messlatte (primär, jede Frage ein eigener Test) und jedes Verfahren gegen jedes (sekundär, mit Holm-Korrektur). Maßgeblich sind die zehn Wiederholungsmittel, die 50 Einzelläufe laufen als gekennzeichnete Gegenprobe mit.
paar() innen · in vergleiche()	Legt eine Zeile der Vergleichstabelle an.
leakage_diagnose()	Prüft, ob es einen messbaren Vorteil bringt, dass die Einstellungen in Wiederholung 0 gesucht wurden. Kostet keine zusätzliche Rechenzeit.
PHASE 5 · SCHLUSSBEWERTUNG	
hold_out()	Die Schlussbewertung: trainiert auf allen 30 Entwicklungsstadtteilen und bewertet auf den 6 zurückgehaltenen, gemeinsam mit beiden Baselines. Als Einstellungen gilt der Satz der Prüfgruppe mit dem kleinsten Fehler in Wiederholung 0.
STEUERUNG	
uebernehmen()	Übernimmt Phase 1 und 2 aus den vorhandenen Ergebnisdateien, statt sie neu zu rechnen. Nur mit dem Zusatz `--weiter`, und mit Abbruch, wenn Daten oder Einstellungen jünger sind als die Ergebnisse.
main() Startpunkt	Startet die Phasen der Reihe nach. Die sechs Hold-out-Stadtteile werden gleich zu Beginn aussortiert, bevor irgendetwas rechnet. Nur mit `holdout` bekommt die Schlussbewertung sie am Ende zu sehen.

modelle/

m03_struktur.py

Der Vergleich für „Welche Art?“

Random Forest und XGBoost gegen die logistische Regression.

LIEST
`klassifikation.parquet` und die Läufe der Messlatte aus v1

SCHREIBT
`results/klassifikation/`: `tuning.csv`, `struktur_folds.csv`, `struktur_mittel.csv`, `vergleich.csv`, `leakage_diagnose.csv`, Vorhersagen, mit `holdout` auch `holdout.csv`

Eine Zielgröße mit vier Einsatzarten, zwei Verfahren, zehn Wiederholungen, fünf Prüfgruppen: 100 Läufe. Der Aufbau spiegelt `m02_menge.py`. Neu sind drei Dinge. Seltene Einsatzarten bekommen mehr Gewicht, statt dass Zeilen kopiert oder gelöscht werden. Die Einsatzarten werden einmal fest in Zahlen übersetzt, nie je Prüfgruppe neu. Und eine Trennfähigkeit, die sich nicht berechnen lässt, bleibt leer, statt durch eine erfundene Zahl ersetzt zu werden.

FUNKTION	WOZU SIE DA IST
BAUEN UND SUCHEN	
`verfahren()`	Baut Random Forest und XGBoost für die vier Einsatzarten, ohne Standardisierung, weil Bäume sie nicht brauchen. Seltene Einsatzarten zählen mehr: beim Random Forest über eine Einstellung, bei XGBoost über Gewichte beim Training.
`suchraum()`	Dieselben Suchräume wie in `m02`, nur ohne die Tweedie-Einstellung, die bei Einsatzarten wirkungslos wäre.
`kodiere()`	Übersetzt die vier Einsatzarten in die Zahlen 0 bis 3, immer in derselben festen Reihenfolge. Würde je Prüfgruppe neu übersetzt, zeigten die Wahrscheinlichkeiten in einer Gruppe ohne Brand auf die falschen Klassen.
`_gewichte()` Hilfsfunktion	Rechnet die Klassengewichte für XGBoost von Hand aus, so wie der Random Forest sie über seine Einstellung bildet.
`tune()`	Sucht die Einstellungen wie in `m02`, bewertet dabei aber mit dem Macro-F1. Mit der einfachen Trefferquote würde die Suche Modelle bevorzugen, die die seltenen Einsatzarten übersehen.
EIN EINZELNER LAUF	
`ein_lauf()`	Ein Training und eine Vorhersage mit Stoppuhr. Liefert Macro-F1, Macro-AUROC, Trefferquote, Laufzeiten und Extrapolationsanteil.
`fitte()` innen · in `ein_lauf()`	Trainiert das Modell mit den passenden Klassengewichten.
`extrapolationsanteil()`	Wie in `m02`. Bewusst doppelt vorhanden, damit `m03` ohne `m02` lauffähig bleibt.

FUNKTION	WOZU SIE DA IST
_gepaart() Hilfsfunktion	Gepaarter Wilcoxon-Test mit Differenz, Konfidenzintervall und gewonnenen Läufen, wortgleich zu `m02`.
_macro_auroc() Hilfsfunktion	Rechnet die Trennfähigkeit über alle vier Klassen. Fehlt im Test eine Klasse, bleibt der Wert leer.
PHASE 1 · EINSTELLUNGEN SUCHEN	
phase_tuning()	Zehn Suchläufe, je Verfahren und Prüfgruppe einer, in Wiederholung 0. Schreibt `tuning.csv`.
_rein_python() Hilfsfunktion	Wandelt NumPy-Zahlen in gewöhnliche Python-Zahlen um, wie in `m02`.
_parameter_je_fold() Hilfsfunktion	Nachschlagetabelle aus `tuning.csv`: Verfahren und Prüfgruppe ergeben die Einstellungen.
PHASE 2 · BEWERTEN	
phase_bewertung()	100 Läufe, jeweils mit einem frischen Modell auf allen Übungsstadtteilen.
PHASE 3 · MITTELN	
aggregiere()	Mittelt zweistufig wie in `m02`.
PHASE 4 · VERGLEICHEN	
vergleiche()	Beide Verfahren gegen die logistische Regression und Random Forest gegen XGBoost. Keine Holm-Korrektur, weil es nur einen einzigen Vergleich der Verfahren untereinander gibt.
paar() innen · in vergleiche()	Legt eine Zeile der Vergleichstabelle an.
leakage_diagnose()	Wie in `m02`, gemessen am Macro-F1.
PHASE 5 · SCHLUSSBEWERTUNG	
hold_out()	Die Schlussbewertung auf den sechs Hold-out-Stadtteilen gegen die logistische Regression aus v1. Als Einstellungen gilt der Satz mit dem höchsten Macro-F1 in Wiederholung 0.
STEUERUNG	
uebernehmen()	Wie in `m02`: übernimmt Phase 1 und 2 nur mit dem Zusatz `--weiter`.
main() Startpunkt	Startet die Phasen der Reihe nach, mit derselben Hold-out-Sperre wie `m02`.

modelle/

m04_shap.py

Welche Merkmale tragen die Vorhersage?

Interpretation: Attribution, Ablation und Verflechtung der Merkmale.

LIEST
beide Analysedateien, die Ergebnisse von m02 und m03 und die Läufe der Messlatte aus v1

SCHREIBT
`results/shap/`: Beiträge, Gruppen, Faktorgruppen, Übersprungenes, Ablationen, Extrapolation, VIF, Verschiebung

Die Datei gehört zur ersten Unterfrage der Arbeit: Welche Merkmalsgruppen hängen mit den Einsätzen zusammen? Mit SHAP erklärt wird dabei nur ein Modell, das seine Messlatte nachweislich schlägt. Ein unterlegenes Modell zu erklären hieße, Rauschen zu erklären. Im Mengenstrang, wo das keines tut, wird stattdessen die Messlatte selbst zerlegt.

Gelesen wird in Gruppen statt in Einzelmerkmalen, weil die Merkmale miteinander verflochten sind und sich ihren Beitrag teilen. Und gerechnet wird auf der Prüfgruppe, deren Stadtteile am seltensten außerhalb des Gelernten liegen.

FUNKTION	WOZU SIE DA IST
`schlagen_die_latte()`	Bestimmt, welche Verfahren ihre Messlatte schlagen. Verlangt werden beide Bedingungen: im Mittel besser und statistisch gesichert. Nur diese werden mit SHAP erklärt, die übrigen stehen mit Begründung in `uebersprungen.csv`.
`ruhigster_fold()`	Sucht die Prüfgruppe mit dem geringsten Extrapolationsanteil in Wiederholung 0. Auf ihr wird die Erklärung gerechnet.
`_beitraege()` Hilfsfunktion	Rechnet je Merkmal den mittleren Beitrag zur Vorhersage: SHAP-Werte bei den Baumverfahren, standardisierte Koeffizienten bei Ridge.
`extrapolation_aufschluesseln()`	Schlüsselt den Extrapolationsanteil nach Merkmal und Stadtteil auf und prüft je Verfahren, ob er mit dem Fehler zusammenhängt.
`ablation_exposition()`	Lässt die Baumverfahren einmal direkt die Anzahl vorhersagen statt der Rate je Einwohner und misst, was die Umrechnung über die Einwohnerzahl bringt. Alles andere bleibt gleich.
`ablation_faktorgruppen()`	Lässt jede Merkmalsgruppe einmal weg und misst, wie viel schlechter die Vorhersage dadurch wird. Im Mengenstrang am Poisson-Modell, im Strukturstrang an den Baumverfahren, mit der logistischen Regression zum Vergleich.
`_ablation_auswerten()` Hilfsfunktion	Verdichtet die Einzelläufe zur mittleren Verschlechterung je Gruppe, zweistufig gemittelt. Ein positiver Wert heißt immer: Weglassen schadet.
`faktorgruppen_baseline()`	Beitrag der Merkmalsgruppen im Mengenstrang, aus den Koeffizienten des Poisson-Modells. Grund: Dort schlägt kein Verfahren die Messlatte, also wird die Messlatte selbst zerlegt.

FUNKTION	WOZU SIE DA IST
`_vif()` Hilfsfunktion	Misst, wie stark die Merkmale miteinander verflochten sind (Varianzinflationsfaktor). Gezählt wird jede Kombination aus Stadtteil und Jahr nur einmal, damit sich der Wert nicht künstlich beruhigt.
`main()` Startpunkt	Führt die sieben Auswertungen nacheinander aus. Bleibt die SHAP-Auswertung leer, weil kein Modell seine Messlatte schlägt, ist das ein Ergebnis und kein Fehler. Mit `--ohne-baeume` läuft die Ablation der Merkmalsgruppen nur für die Baselines und deutlich schneller.
`verschiebung()`	Wie weit liegen die Mengenvorhersagen im Mittel daneben, und in welche Richtung? Rechnet aus den gespeicherten Vorhersagen von `m02` je Verfahren die mittlere Verschiebung (negativ heißt zu tief), den Fehler und den Fehler ohne diese Verschiebung. Anlass: Ridge rechnet auf log(1+y), und die Rückrechnung liegt im Mittel zu tief. Liest nur, passt kein Modell an.

modelle/

parametersensitivitaet.py

Hängt das Ergebnis am Einstellungssatz?

Kreuzprobe: jede Prüfgruppe mit jedem der fünf gefundenen Einstellungssätze.

LIEST
beide Analysedateien, `tuning.csv` und die Laufdateien aus m02 und m03

SCHREIBT
`results/parametersensitivitaet/`: `matrix.csv`, `zusammenfassung.csv`, `spannen.csv`, `bericht.md`

Die Suche liefert je Verfahren fünf Einstellungssätze, einen je Prüfgruppe. In der Schlussbewertung tritt aber nur einer davon an. Diese Datei misst, wie viel diese Wahl ausmacht: Ist der Unterschied zwischen eigenem und fremdem Satz klein gegen den Unterschied zwischen den Verfahren, trägt die Schlussbewertung. Ist er groß, hängt sie zum Teil vom Zufall der Wahl ab (Band 2, Teil 10).

Alles läuft innerhalb der 30 Entwicklungsstadtteile. Das Hold-out bleibt gesperrt, und es entsteht keine zweite Schlussbewertung.

FUNKTION	WOZU SIE DA IST
`parametersaetze()`	Liest die fünf Einstellungssätze je Verfahren aus `tuning.csv`, im Mengenstrang jeden Satz nur einmal.
`lage_im_suchraum()` Hilfsfunktion	Rechnet aus, wo ein gewählter Wert in seinem Suchraum liegt: 0 an der Untergrenze, 1 an der Obergrenze.
`spannen()`	Wie weit liegen die fünf gewählten Werte je Einstellung auseinander? 0 heißt, alle Prüfgruppen wählen dasselbe, 1 heißt, die Wahl reicht von Rand zu Rand.
`kreuzprobe()`	Bewertet jede Prüfgruppe mit jedem der fünf Sätze, nur in Wiederholung 0. Die Diagonale ist der eigene Satz, die übrigen 20 Zellen sind fremde.

FUNKTION	WOZU SIE DA IST
zusammenfassung()	Stellt je Verfahren den eigenen gegen die fremden Sätze: Wie viel schlechter ist ein fremder im Mittel, und wie oft war ein fremder sogar besser?
_kontrolle() Hilfsfunktion	Prüft, dass die Diagonale genau die Werte des Hauptlaufs wiederholt. Weicht sie ab, hat sich an Aufteilung oder Modell etwas verändert, und das Skript warnt.
bericht()	Setzt die Lesefassung `bericht.md` zusammen. Hier wird nichts gerechnet.
md() innen · in bericht()	Wandelt eine Tabelle in eine Texttabelle um.
main() Startpunkt	Rechnet Spannen und Kreuzprobe für beide Stränge oder nur für einen davon und schreibt die vier Dateien.

modelle/

suchdiagnose.py

War die Suche groß genug?

Wiederholt die Suche und schreibt jede einzelne Ziehung mit.

LIEST
beide Analysedateien

SCHREIBT
`results/suchdiagnose/kurve.csv` und `zusammenfassung.md`, mit `--test` nach `results/suchdiagnose_test/`

`tuning.csv` hält nur den Gewinner jeder Suche fest, nicht den Weg dorthin. Diese Datei wiederholt die Suche des Hauptlaufs mit denselben Suchräumen, Prüfgruppen und Startwerten und hält jede Ziehung fest. Daraus entsteht eine Suchkurve: der beste Wert nach n Ziehungen. Steigt sie in der zweiten Hälfte kaum noch, war das Budget nicht zu klein.

Wichtig ist auch, was die Datei nicht sagt: Ein besserer Wert innerhalb der Suche bedeutet noch keine bessere Vorhersage für unbekannte Stadtteile. Die Ergebnisse von `m02` und `m03` bleiben unberührt, das Hold-out wird nicht gelesen.

FUNKTION	WOZU SIE DA IST
suchraum()	Holt den Suchraum eines Verfahrens, genau wie im Hauptlauf, in der Klassifikation ohne die Tweedie-Einstellung.
_verteilungen() Hilfsfunktion	Übersetzt den Suchraum in Zufallsverteilungen, wortgleich zu `m02`.
eine_suche()	Ein Suchlauf mit Budget 100, der jede einzelne Ziehung mit ihrem Gütewert zurückgibt, nicht nur den Sieger.
_md() Hilfsfunktion	Baut eine Texttabelle von Hand, ohne Zusatzpaket.

FUNKTION	WOZU SIE DA IST
`kurve()`	Macht aus den Ziehungen die Suchkurve: der beste Wert nach 1, 2, 3 und so weiter Ziehungen.
`main()` Startpunkt	Wiederholt die Suche für alle Verfahren und Prüfgruppen, schreibt die Suchkurve und vergleicht den Gewinn der zweiten fünfzig Ziehungen mit der Streuung zwischen den Prüfgruppen. Mit `--test` als kurzer Probelauf mit Budget 6.

modelle/

trennschaerfe.py

Hätte der Test einen Unterschied erkannt?

Trennschärfe des gepaarten Tests bei zehn Wiederholungsmitteln.

LIEST
`vergleich.csv` aus m02 und m03

SCHREIBT
`results/trennschaerfe/trennschaerfe.csv` und `raster.csv`

Ein nicht signifikanter Vergleich heißt nicht, dass zwei Verfahren gleich gut sind (Band 1, Teil 2). Er sagt erst etwas, wenn feststeht, welche Unterschiede dieser Test bei nur zehn Werten überhaupt erkennen kann. Die Datei rechnet das durch Simulation aus: Sie zieht viele künstliche Stichproben mit einem bekannten Unterschied und zählt, wie oft der Test anschlägt.

FUNKTION	WOZU SIE DA IST
`effektstaerke()`	Rechnet aus einer Zeile der Vergleichstabelle die Effektstärke d: mittlere Differenz geteilt durch die Streuung der Differenzen.
`trennschaerfe()`	Schätzt, in welchem Anteil der simulierten Stichproben der Wilcoxon-Test bei dieser Effektstärke einen Unterschied findet.
`run()` Startpunkt	Rechnet die Trennschärfe für jeden tatsächlichen Vergleich und für eine Reihe von Effektstärken von 0,1 bis 1,6 und schreibt beide Tabellen.

TEIL 4

In drei Sätzen

> Was man über diesen Ordner wissen muss, wenn man nur eine Minute hat.

1. `m02_menge.py` und `m03_struktur.py` führen den Vergleich nach demselben Muster in vier Phasen, die Schlussbewertung kommt nur mit `holdout` dazu.

2. `m04_shap.py` erklärt mit SHAP nur Modelle, die ihre Messlatte schlagen, und fragt neben der Aufmerksamkeit auch, was das Weglassen einer Merkmalsgruppe kostet.

3. `suchdiagnose.py`, `parametersensitivitaet.py` und `trennschaerfe.py` prüfen die Suche, die Wahl der Einstellungen und den Test und verändern dabei kein Ergebnis des Vergleichs.